

Solving Benchmark Security Problems

with an Agentic Penetration Tester

How the agent works in general

Before the six types, here is the basic way the agent operates, because every type reuses the same loop.

The agent runs inside a Kali Linux environment, the same setup a human tester would use. It works in a simple loop: it looks at the task, decides on a command, runs that command in the shell, reads the result, and then decides the next command. It keeps doing this until it finds the secret value, called the flag.

Across almost every task the agent follows five stages:

1. Reconnaissance, gather information about the target.
2. Scanning, find open ports, services, or useful files.
3. Exploitation, use the weakness to get in or get the data.
4. Escalation, gain higher access if needed.
5. Capture, read and submit the flag.

The six problem types below are just different kinds of weakness the agent meets during stage 3.

1. Crypto (cryptography)

The problem. The target hides a secret behind a code or cipher. The flag is locked by some form of encryption, and the encryption is weak or used wrongly.

How the agent solves it. The agent first studies the crypto service or file to understand which algorithm is used and how. It looks for the common mistakes, for example a key that is too small, a reused key, or a weak random number generator. Once it spots the weakness, it writes and runs a small script that performs the matching attack to recover the key or decrypt the secret.

Tools the agent uses. Python with crypto libraries like pycryptodome, and SageMath for the heavier maths.

Honest limit. Crypto is one of the harder types for an agent, because it often needs real mathematical insight, not just running a tool.

2. Pwn (binary exploitation)

The problem. The target is a compiled program with a memory bug. The agent has to abuse that bug to take control of the program and make it reveal the flag.

How the agent solves it. The agent first studies the program to find the bug, for example a place where the program accepts more input than it should. It then builds a carefully shaped input that triggers the bug and redirects the program to do what the agent wants. It tests this against the running program and adjusts until it works.

Tools the agent uses. pwntools to build the attack input, gdb to watch the program run, and Ghidra to study the code.

Honest limit. Pwn is the hardest type for an agent, because it needs many rounds of trial and error with a debugger and very precise control of memory.

3. Rev (reverse engineering)

The problem. The target is a compiled program that checks for a secret. The agent has to read the program's hidden logic and work out what input it wants. Nothing is broken here, it is a puzzle of understanding.

How the agent solves it. The agent opens the program in a decompiler and reads how it checks the input. It traces the logic backwards to figure out which value would pass the check. Sometimes it runs the program step by step to confirm what each part does, then works out the flag.

Tools the agent uses. Ghidra and IDA to turn the program back into readable code, gdb to run it step by step, and strings to pull out readable text.

Honest limit. Rev is hard for an agent because it needs long, careful reasoning across a large program, and the agent can lose the thread partway.

4. Web (web exploitation)

The problem. The target is a website or web service with a flaw in how it handles input or access. The agent has to attack that flaw to reach the flag.

How the agent solves it. The agent first explores the site to map its pages and inputs. It then tests each input for the common web flaws, such as SQL injection, cross-site scripting, broken access control, or file inclusion. When it finds a flaw that works, it uses it to read the hidden file or gain admin access, then captures the flag.

Tools the agent uses. Burp Suite or curl to inspect and change requests, sqlmap for SQL injection, and ffuf or gobuster to find hidden pages.

Honest limit. Web tasks that chain two flaws together, or that need the agent to hold state across many requests, are the harder ones.

5. Forensics

The problem. The flag is hidden inside files or data, for example an image, a network capture, or a memory dump. Nothing needs to be exploited. The agent has to investigate and recover the hidden information.

How the agent solves it. The agent first identifies what kind of artifact it has. For a network capture it follows the traffic to find the hidden data. For an image it checks for data hidden inside it. For a memory dump it pulls out processes, files, or keys. It keeps digging until the flag appears.

Tools the agent uses. Wireshark and tcpdump for traffic, binwalk and strings for files, and Volatility for memory dumps.

Honest limit. Large artifacts, like big memory images, are hard because they do not fit in the agent's context and need the right specialist tool.

6. Misc (miscellaneous)

The problem. These are mixed or unusual tasks that do not fit the other five types. They might be a scripting puzzle, an escape from a restricted environment, or a task that combines several skills.

How the agent solves it. Because these vary, the agent first has to work out what kind of problem it is even looking at. Then it applies the right mix of skills, often writing a small script to automate the solution or to interact with the service in an unusual way.

Tools the agent uses. Python and standard command line tools, plus CyberChef for quick decoding.

Honest limit. The main difficulty is that the agent has to figure out the problem type on its own before it can even start, which adds an extra step.

